IPEDS App Deployment Plan

Redesigned app | Working plan | October 26 2026

This document records how we can deploy the redesigned IPEDS Research Companion for other people to use. It preserves the implementation sequence, launch checks and unresolved concerns so we can resume without reconstructing the earlier discussion.

Recommendation: use a new GitHub repository, Streamlit Community Cloud and a private Cloudflare R2 Standard bucket for a limited pilot. Keep the existing graduation-rate prototype and its URL unchanged. The design is feasible; cloud implementation and hosted capacity testing are still pending.

Current status

The owner has GitHub and Streamlit accounts. The local code and data were reviewed, and current hosting documentation was checked. R2 configuration, uploads, a clean Linux build and remote deployment have not been completed. This plan does not establish production capacity or set a public launch date.

Proposed services and initial costs

Service	Purpose	Expected initial cost
GitHub	Code, configuration, small assets and data manifest	Existing account
Streamlit Community Cloud	Runs the interface, SQLite queries and Claude requests	Free hosting [1]
R2 Standard	Durable copies of the versioned database release	Potentially $0 within allowances [2]
Anthropic API	Prompt interpretation and source-backed planning	Usage based; set a spending limit [3]

Data and operating model

The candidate runtime release is 13 SQLite databases totaling 3,057,135,616 bytes: 3.057 GB or 2.847 GiB. It contains ten annual files plus metadata, analysis and unit catalogs. Keep these large files outside ordinary Git source control.

The Streamlit server downloads the files from R2, verifies them and queries local copies. R2 stores files; it does not execute the SQLite queries. Each visitor receives answers, charts and requested downloads rather than the full database package.

R2 currently includes 10 GB-month Standard storage, one million Class A and ten million Class B operations monthly, with free direct egress. The current release fits if other account usage leaves room. Storage above the allowance is $0.015 per GB-month; requests can add charges. An R2 subscription must still be enabled. Recheck prices before setup. [2]

Deployment implementation

Phase 1 Prepare a separate release package

Create a clean repository for the redesign. The current local workspace still points to the original graduation-rate Git remote, so do not push the entire workspace into that deployment.

- Include prototype, analysis_catalog, the reviewed measure registry, required JSON and JavaScript assets, root Streamlit configuration and declared dependencies. Use prototype/app.py as the entry point.
- Exclude local .packages, environments, credentials, raw Access files, conversion outputs, proposal logs and test artifacts. Preserve the existing relative directory layout.
- Create a manifest of database object keys, relative paths, sizes and SHA-256 checksums. Pair the code revision with one specific data release.

Output: a reviewed code bundle and manifest. Static inspection suggests field_semantics.sqlite is a preparation artifact rather than a runtime dependency; confirm its exclusion in a clean-package smoke test. Nothing should be deleted as part of packaging.

Phase 2 Configure R2 and publish the data release

- Enable R2 Standard storage and create a private bucket. Upload the 13 reviewed runtime databases using versioned object paths.
- Use separate credentials for uploading and app access. Give the hosted app bucket-scoped Object Read only access through the S3-compatible API. [4]
- Record the bucket, release identifier and manifest location. Keep credentials in hosted secrets, never in GitHub or browser code.

Phase 3 Add reliable data initialization

Add the downloader before the UI checks for its local metadata catalog. There is no R2 downloader in the current app. Add and pin the selected Python S3 client dependency.

- Stream downloads to disk, bound parallel downloads and guard initialization across simultaneous sessions. Use temporary files, checksum verification and atomic completion.
- Reuse a completed matching release across sessions and reruns. Show clear progress and recover from interrupted downloads or missing files without serving mismatched catalogs.
- Start with the complete data package. Current coverage and guide inventory inspect all annual sources; fetching just one year would require more changes.

Phase 4 Configure hosted execution and controls

Use Python 3.12 as the initial clean-build target, matching the local major/minor version. Install declared packages without Windows binaries or OneDrive paths. Verify Linux file-name case, SQLite full-text support and chart/image exports.

Set server-side Anthropic and R2 secrets. Use a dedicated Anthropic workspace/key with spend and rate limits. Add application request limits, bounded concurrent model work and an operator AI cutoff. Keep Automatic as the public model default: Haiku for interpretation/reviewed planning, Sonnet for catalog planning/writing. Restrict development tools and model overrides to server configuration or authorized operators. [3, 5]

Validation and launch

Phase 5 Deploy a preview and measure it

Connect the new repository to a separate Streamlit Community Cloud app, configure secrets and deploy a preview URL. Keep the original app and credentials separate. Start with one or two active users, then test five active sessions; these are test levels, not established capacity commitments.

- Measure first deployment, first data download, warm requests, restart and wake behavior. Verify that completed files are reused and missing files can be restored.
- Measure actual Linux/container memory and CPU, request latency, model calls, token cost and errors. Include long conversations, national analyses and simultaneous Excel downloads.
- Check general data/year questions, Ivy comparisons, unavailable Fall 2025 requests, definitions, categorical lookups, graduate percentages, mixed requests, chart edits and source citations.
- Verify session separation, model/API failures, rate-limit messages, refresh behavior and handling of interrupted downloads or invalid checksums.

Evidence already available

The October 23 Windows probe observed approximately 202 MiB peak resident memory and 539 MiB sampled private committed memory. These measures are different and must not be added. Two retained UI test sessions plus simultaneous backend workloads completed, but this was not a two-browser hosted load test and included no Claude calls or R2 transfer.

That probe predates the latest conversation, guide and task-tracking features. It supports trying a small pilot, while current hosted measurements must determine operating capacity. The data package is held on disk rather than loaded wholesale into RAM.

Streamlit currently lists approximate resource limits of 0.078–2 CPU cores, 690 MB–2.7 GB RAM and up to 50 GB disk, with figures still labeled February 2024. They are variable limits rather than reserved resources. Apps hibernate after 12 hours without traffic. Local-file persistence is not guaranteed. [6, 7]

Phase 6 Release a limited public pilot

Release only after the preview reproduces expected answers, verifies every database, respects usage controls and passes the agreed workload. Decide acceptable response times, concurrency and spending before judging these tests. Keep known unsupported calculations and benchmark limitations visible.

- Record the deployed code revision, data release, dependency versions, operator settings and launch checks.
- Explain that chats are temporary, which information is sent to Anthropic and whether any questions are retained. Do not add durable question logging without a retention decision.
- Review costs and errors during the pilot before expanding access. Claude usage is expected to be the main variable expense; there is no dependable fixed price per question yet.

Rollback and fallback hosting

Retain the previous code revision and matching data manifest. Roll back both together; disable new AI requests if cost or reliability fails. If free hosting cannot sustain the measured workload or startup time, use a host with explicitly allocated CPU/RAM and persistent disk. SQLite and R2 can remain in the architecture.

Areas of concern

Resolve these items before the relevant launch stage. Their importance is based on the current implementation and deployment design; the list is not evidence that any failure has occurred on Streamlit.

Concern	Why it matters	Response and verification
Cold starts and storage	About 3 GB may need restoring. Host-local files are not durable.	Measure initial download and wake/restart behavior. Cache verified files, restore from R2 and recover partial transfers.
Shared memory and CPU	Free-host capacity varies. National queries, large histories and concurrent exports can raise peaks.	Test current code with real sessions. Bound work and monitor usage; move hosts if measured headroom is inadequate.
Public API spending	Visitors share a paid key. Session limits can be bypassed by recreating sessions.	Set provider workspace spend/rate limits, app request limits, bounded concurrency and an operator cutoff.
Developer controls	The current About page lets every visitor enable tools or select a more expensive model.	Treat the switch as a UI convenience, not authorization. Enforce public settings on the server; test visitor access.
Release consistency	A partial or mixed release can invalidate catalogs or produce incorrect bindings.	Use a versioned checksum manifest, atomic completion and code/data pairing. Fail clearly on mismatch.
User information	Proposal JSON can contain question/context data; local files can accumulate or disappear.	Choose temporary versus durable retention. Bound writes, keep credentials out of logs and document any collection.
Linux portability	The current development environment uses Windows and project-local installed packages.	Run a clean dependency install and source/export smoke tests without local binaries or absolute Windows paths.
Analytical reliability	Passing small tests does not establish accuracy for arbitrary IPEDS questions.	Retain source, population and formula validation, explicit uncertainty and unsupported-query safeguards. Expand pilot cases.

What is not established yet

No Linux cloud build, R2 transfer benchmark, deployed memory measurement or hosted concurrency test has been completed. Hosting feasibility should not be presented as a guarantee of uptime, user capacity or universal analytical accuracy.

Decisions and where to resume

Decisions still needed

- Pilot audience: invited users first, or anonymous public access with appropriate usage controls.
- Spending: monthly Anthropic cap, request allowance, concurrency limit and who monitors costs and errors.
- Ownership: new GitHub repository, Streamlit app name/URL and R2 bucket under the intended accounts.
- Retention: whether proposals, feedback or questions should be kept, for how long and in which durable store.
- Acceptance targets: acceptable cold-start time, warm response time and concurrent workload before public launch.

Resume from this point

Next implementation task: prepare the separate code bundle and data manifest, then build the R2 bootstrap and public-use controls. Account setup and upload follow the reviewed release package. No application or hosting configuration was changed during the feasibility investigation.

Project folder: C:/Users/liyuj/OneDrive/2026/IPEDS Project/agent-redesign

Supporting files below are relative to that folder. This Word plan records the October 26 position; older deployment notes are historical context.

- DEPLOYMENT_FEASIBILITY_2026-10-26.md — detailed investigation and source links.
- evaluations/deployment_feasibility_inventory.json — exact candidate database paths and sizes.
- LOCAL_PERFORMANCE_REPORT.md — October 23 local test method, results and limits.
- evaluations/capacity_local_results.json — underlying local performance observations.
- DEPLOYMENT_PLAN.md and STREAMLIT_CAPACITY_REVIEW.md — earlier planning notes.

Official references checked October 26 2026

1 Streamlit Community Cloud

2 Cloudflare R2 pricing

3 Anthropic workspace spending and rate controls

4 R2 authentication and token permissions

5 Streamlit hosted secrets

6 Streamlit resources and hibernation

7 Streamlit local storage qualification

8 Streamlit repository and dependency layout

Recheck service prices, allowances and hosting limits when resuming. They can change after this review.